

LangChain and LangGraph Complete Guide

Introduction

Modern AI applications are no longer simple prompt-and-response systems. They interact with databases, retrieve documents, call APIs, execute tools, remember conversations, and perform multi-step reasoning.

LangChain and LangGraph are two popular frameworks that help developers build such intelligent AI systems.

Examples include:

- AI Chatbots
 - RAG Systems
 - Customer Support Assistants
 - Research Agents
 - Coding Assistants
 - LMS Learning Assistants
 - Enterprise Copilots
-

Chapter 1: What is LangChain?

LangChain is an open-source framework designed for building applications powered by Large Language Models (LLMs).

Core capabilities:

- Prompt Management
- Document Loading
- Text Chunking
- Embeddings
- Vector Databases
- Retrieval
- Memory
- Tool Calling
- Agent Development

Instead of manually writing complex workflows, LangChain provides reusable components.

Chapter 2: Why LangChain?

Without LangChain:

```
```text id="lc_1" LLM ↓ Manual Prompting ↓ Custom Retrieval ↓ Custom Tools ↓ Custom Memory
```

With LangChain:

```
```text id="lc_2"
LLM
↓
LangChain Components
↓
Production Application
```

Benefits:

- Faster development
- Reusable abstractions
- Easier maintenance
- Large ecosystem

Chapter 3: Installing LangChain

Install LangChain:

```
```bash id="lc_install" pip install langchain
```

Install integrations:

```
```bash id="lc_install2"
pip install langchain-community
```

Install OpenAI integration:

```
```bash id="lc_install3" pip install langchain-openai
```

Install Google integration:

```
```bash id="lc_install4"
pip install langchain-google-genai
```

Chapter 4: Language Models

A language model is the reasoning engine.

Example:

```
```python id="lc_model" from langchain_openai import ChatOpenAI
```

```
model = ChatOpenAI()
```

Google Gemini:

```
```python id="lc_gemini"
from langchain_google_genai import ChatGoogleGenerativeAI
```

Examples:

- GPT
- Gemini
- Claude
- Llama
- Qwen
- Mistral

Chapter 5: Prompts

Prompts control model behavior.

Simple example:

```
```python id="lc_prompt" prompt = "Explain React Hooks"
```

Prompt templates:

```
```python id="lc_prompt2"
from langchain.prompts import PromptTemplate
```

```
```python id="lc_prompt3" template = """ Explain {topic} """
```

Benefits:

- Reusability
- Dynamic content
- Cleaner code

---

# Chapter 6: Chains

Chains connect multiple operations together.

Example:

```
```text id="chain_flow"
```

Input

↓

Prompt

↓

LLM

↓

Output

Use cases:

- Content generation
- Summarization
- Classification

Chains simplify workflow creation.

Chapter 7: Document Loaders

Document loaders read external files.

Supported formats:

- PDF

- DOCX
- HTML
- TXT
- CSV

Example:

```
```python id="loader_pdf" from langchain_community.document_loaders import PyPDFLoader
```

```
```python id="loader_pdf2"  
loader = PyPDFLoader("react_notes.pdf")  
docs = loader.load()
```

Applications:

- LMS documents
 - Company knowledge bases
 - Research papers
-

Chapter 8: Text Splitters

Large documents must be split into chunks.

Example:

```
```python id="splitter1" from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
```python id="splitter2"  
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=100  
)
```

Benefits:

- Better retrieval
 - Reduced token usage
-

Chapter 9: Embeddings

Embeddings convert text into vectors.

Example:

```
```text id="embed_flow" Text ↓ Embedding Model ↓ Vector
```

Popular models:

- OpenAI Embeddings
- BGE
- Gemini Embeddings
- Sentence Transformers

Embeddings power semantic search.

---

# Chapter 10: Vector Stores

Vector stores store embeddings.

Supported databases:

- Pinecone
- Qdrant
- Chroma
- Weaviate
- Milvus
- pgvector

Example:

```
```python id="vector1"
from langchain.vectorstores import Chroma
```

Purpose:

- Similarity search
- Knowledge retrieval

Chapter 11: Retrievers

Retrievers find relevant documents.

Workflow:

```text id="retriever\_flow" Question ↓ Embedding ↓ Similarity Search ↓ Relevant Chunks

Example:

```
```python id="retriever_code"
retriever = vectorstore.as_retriever()
```

Applications:

- LMS chatbots
- Enterprise search
- Knowledge assistants

Chapter 12: Building a RAG Pipeline

Basic architecture:

```text id="rag\_arch" Documents ↓ Chunking ↓ Embeddings ↓ Vector Database ↓ Retriever ↓ LLM ↓ Answer

Benefits:

- More accurate responses
- Less hallucination
- Private knowledge access

---

# Chapter 13: Memory

Memory helps AI remember conversations.

Without memory:

```
```text id="mem_no"
Every message is isolated
```

With memory:

```
```text id="mem_yes" Conversation Context ↓ Better Responses
```

Types:

- Buffer Memory
- Summary Memory
- Vector Memory

---

## # Chapter 14: Tools

Tools allow models to interact with external systems.

Examples:

- Calculator
- Database Query
- Search Engine
- API Call

Example:

```
```python id="tool_example"
def get_weather(city):
    return "Sunny"
```

The model can call tools when needed.

Chapter 15: Agents

Agents decide which actions to take.

Traditional chatbot:

```
```text id="simple_bot" Question ↓ Answer
```

Agent:

```
```text id="agent_flow"
```

Question

↓

Reason

↓

Choose Tool

↓

Execute Tool

↓

Answer

Agents are significantly more capable.

Chapter 16: Agent Architectures

Popular patterns:

ReAct

```
```text id="react_pattern" Reason Act Observe Repeat
```

```
Tool Calling
```

```
```text id="tool_call_pattern"
```

Question

↓

Tool

↓

Result

↓

Answer

Planner-Executor

```
```text id="planner_executor" Planner ↓ Tasks ↓ Executor
```

```

```

```
Chapter 17: Multi-Agent Systems
```

Multiple agents collaborate.

Example:

```
```text id="multi_agent"
Research Agent
↓
Analysis Agent
↓
Writing Agent
```

Applications:

- Business intelligence
- Research automation
- Complex workflows

Chapter 18: What is LangGraph?

LangGraph is a framework built for stateful AI workflows.

It extends LangChain by providing:

- State Management
- Cycles
- Multi-Agent Workflows
- Complex Reasoning

Think of LangGraph as:

```
```text id="graph_desc" LangChain + Workflow Engine
```

```

Chapter 19: Graph Concepts

LangGraph consists of:

Nodes

Individual tasks.
```

Examples:

- Retrieval
- Search
- Tool Calling

### Edges

Connections between nodes.

### State

Shared information across workflow steps.

---

# Chapter 20: LangGraph Workflow

Example:

```
```text id="graph_flow"
User Question
    ↓
Retrieve Docs
    ↓
Analyze Context
    ↓
Call Tool
    ↓
Generate Answer
```

Each node performs a specific task.

Chapter 21: State Management

State stores information throughout execution.

Example:

```
```json id="state_example" { "question": "...", "documents": [], "answer": "" }
```

Benefits:

- Persistent context
- Better reasoning
- Easier debugging

---

## # Chapter 22: Building an LMS Agent

Workflow:

```
```text id="lms_agent"
```

Student Question

↓

Check Role

↓

Retrieve Documents

↓

Query Database

↓

Generate Answer

Examples:

Student:

```
```text id="student_query" Show my pending assignments
```

Teacher:

```
```text id="teacher_query"
```

Show attendance report

Admin:

```
```text id="admin_query" Show enrollment statistics
```

---

## # Chapter 23: Tool Calling with Databases

Agents can query databases.

Example:

```
```text id="db_flow"
Question
↓
SQL Query
↓
Database
↓
Result
↓
Answer
```

Applications:

- Student records
- Attendance
- Course information

Chapter 24: Human-in-the-Loop Systems

Some workflows require approval.

Example:

```
```text id="human_loop" AI Draft ↓ Human Review ↓ Final Action
```

Applications:

- Finance
- Healthcare
- Legal systems

---

# Chapter 25: Best Practices

1. Keep tools focused.
2. Use structured outputs.
3. Validate tool inputs.
4. Implement access control.
5. Use role-based retrieval.
6. Monitor token usage.
7. Log agent actions.
8. Add fallback responses.

9. Evaluate retrieval quality.
10. Secure sensitive data.

---

## # Chapter 26: Production Architecture

Enterprise architecture:

```
```text id="prod_arch"
Frontend
  ↓
API Layer
  ↓
LangGraph Agent
  ↓
Retriever
  ↓
Vector Database
  ↓
LLM
```

Additional services:

- Redis
- PostgreSQL
- Monitoring
- Logging

Summary

LangChain and LangGraph are foundational frameworks for building modern AI applications. LangChain provides reusable components for prompts, retrieval, embeddings, memory, tools, and agents. LangGraph extends these capabilities with stateful workflows, multi-agent systems, and advanced orchestration. Together they enable developers to build powerful AI assistants, LMS chatbots, enterprise copilots, and autonomous agent systems capable of reasoning, retrieval, and action.